Rethinking Broken Object Level Authorization Attacks Under Zero Trust Principle

ANBIN WU and ZHIYONG FENG, The College of Intelligence and Computing, Tianjin University, CHINA
RUITAO FENG*, Faculty of Science and Engineering, Southern Cross University, Australia
ZHENCHANG XING, CSIRO Data61, Australia
YANG LIU, School of Computer Science and Engineering, Nanyang Technological University, Singapore

RESTful APIs facilitate data exchange between applications, but they also expose sensitive resources to potential exploitation. Broken Object Level Authorization (BOLA) is the top vulnerability in the OWASP API Security Top 10, exemplifies a critical access control flaw where attackers manipulate API parameters to gain unauthorized access. To address this, we propose BolaZ, a defense framework grounded in zero trust principles. BolaZ analyzes the data flow of resource IDs, pinpointing BOLA attack injection points and determining the associated authorization intervals to prevent horizontal privilege escalation. Our approach leverages static taint tracking to categorize APIs into producers and consumers based on how they handle resource IDs. By mapping the propagation paths of resource IDs, BolaZ captures the context in which these IDs are produced and consumed, allowing for precise identification of authorization boundaries. Unlike defense methods based on common authorization models, BolaZ is the first authorization-guided method that adapts defense rules based on the system's best-practice authorization logic. We validate BolaZ through empirical research on 10 GitHub projects. The results demonstrate BolaZ's effectiveness in defending against vulnerabilities collected from CVE and discovering 35 new BOLA vulnerabilities in the wild, demonstrating its practicality in real-world deployments.

CCS Concepts: • **Software and its engineering → Software defect analysis**; • **Security and privacy → Web application security**.

Additional Key Words and Phrases: API security, BOLA attack, RESTful APIs, Access control.

1 Introduction

RESTful APIs have become the standard for accessing web-oriented resources, enabling users to initiate operational requests through HTTP methods, paths, and parameters. However, the parameters of RESTful APIs are user-controlled, hence, inherently untrusted. Attackers can exploit this by tampering with the ***resource ID*** parameter to access sensitive data of other users without authorization, leading to a **Broken Object Level Authorization (BOLA) attack** [19]. Resource ID is the unique identifier of resources in RESTful APIs that are used to operate (INSERT, DELETE, UPDATE, READ) resources, that is, **BOLA attack injection point**. The vulnerability arises from over-reliance on the user-supplied resource ID, without enforcing proper access control mechanisms.

*Corresponding author

Authors' Contact Information: Anbin Wu, wuanbin@tju.edu.cn; Zhiyong Feng, zyfeng@tju.edu.cn, The College of Intelligence and Computing, Tianjin University, CHINA; Ruitao Feng, Faculty of Science and Engineering, Southern Cross University, Australia, ruitao.feng@scu.edu.au; Zhenchang Xing, CSIRO Data61, Australia, zhenchang.xing@anu.edu.au; Yang Liu, School of Computer Science and Engineering, Nanyang Technological University, Singapore, yangliu@ntu.edu.sg.

Just Accepted

The BOLA vulnerability is an access control vulnerability, and developers need to perform some security checks before users operate sensitive information to defend against the vulnerability. BOLA vulnerabilities are derived from the lack of object-level authorization checking [18]. To accurately detect BOLA vulnerabilities, it is necessary to understand the access control policy of resources. Through the study of the existing work, there are two main ways to obtain access control policies. 1). Manually provide authorization policies rules [14], [31], [5]. This method relies on manually labeling authorization rules for each resource operation, which is tedious and prone to errors [17]. 2). Deduce the authorization model of resources through source code [37], [29], [18], [32]. This type of method first artificially summarizes the application's resource authorization model, and infers which authorization model the resource belongs to by analyzing the source code. However, this type of method is based on the authorization model of human summary. When the system authorization model is inconsistent with the authorization model of human summary, it will lead to false positives. The authorization model's static nature and the unknown system logic's dynamic nature are contradictory.

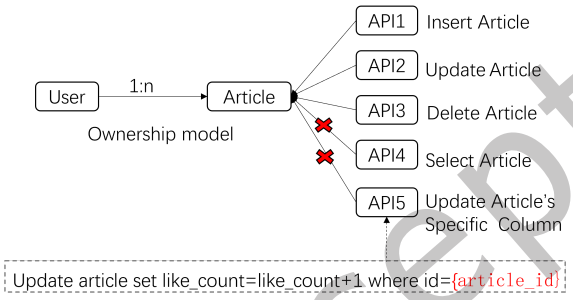


Fig. 1. BOLARAY authorization model

BOLARAY (CCS'24, Oct) [18] is the state-of-the-art (SOTA) method for detecting BOLA attacks. By analyzing real-world BOLA vulnerabilities in open-source applications, BOLARAY identifies four common object-level authorization models. However, these models, being manually summarized, lack adaptability to the diverse and evolving nature of modern web applications. As illustrated in Fig.1, BOLARAY classifies the *Article* table under the ownership model, where an article can only be updated or deleted by its owner. This model fails to support SELECT operations. For instance, API5 in Fig.1 involves an UPDATE statement that allows users to add likes to articles. However, BOLARAY incorrectly assumes that articles can only be updated by their owners. This rigid interpretation overlooks the actual propagation patterns of resource identifiers in web applications [1], leading to semantic mismatches. As a result, such out-of-context authorization logic limits BOLARAY's ability to comprehensively detect BOLA vulnerabilities.

We revisit the essence of BOLA attacks—horizontal privilege escalation and reconceptualize their defense as a problem of minimally scoped authorization. Specifically, defending against BOLA attacks involves isolating the injection point (i.e., the resource ID) within the narrowest permission boundary that aligns with the application's logic. The resource ID parameter functions dually as an attack surface vulnerable to horizontal privilege escalation [36], and as a protection surface responsible for enforcing least privilege access control [7]. Effective mitigation requires accurately determining the authorization interval for each resource ID, ensuring users can only access or manipulate resources within their minimum necessary permissions [20].

Zero Trust, grounded in the principle of "never trust, always verify" [35], emphasizes fine-grained, identity-based access control to mitigate risks associated with unauthorized lateral movement. ***Micro-segmentation (MSG)***, a crucial component of zero trust, creates a security closed-loop by considering multiple factors—such as user roles, processes, and access contexts—to define the minimal security boundary for resource IDs based on

Just Accepted

API context data flow. This not only aligns with the API's authorization logic but also enforces the principle of minimum permissions, making MSG an effective strategy for defending against BOLA attacks.

This paper introduces BolaZ[1], a novel framework designed to defend against BOLA attacks from two perspectives: attack surface discovery and protection surface authorization. BolaZ leverages an attacker's viewpoint to analyze BOLA attack data flows, accurately **identifying the BOLA attack injection points**. BolaZ takes advantage of the context relationship between API parameters and resource ID to compensate for the shortcomings of identifying injection points based on fixed authorization models. Additionally, BolaZ divides resources into smaller logical intervals based on resource ID workflows, isolating the protection surface to the **minimal authorization intervals** defined by the system's authorization logic. This approach enables effective resource isolation across different users and business processes, overcoming the limitations of static authorization models.

Through empirical research on 10 GitHub projects verified to contain RESTful API usage, we first quantitatively evaluate BolaZ's effectiveness by assessing the precision and recall of attack injection point identification, authorization interval determination, and performance overhead. Our experimental selection criteria—GitHub stars, application type, and logic control type (detailed in Section 3.4.2)—ensure a comprehensive coverage of various application scenarios. Next, we collect relevant vulnerabilities from the Common Vulnerabilities and Exposures (CVE) DB based on three BOLA attack modes (detailed in Section 3.1) to demonstrate BolaZ's capability in detecting these vulnerabilities. Finally, we apply BolaZ to scan for potential BOLA vulnerabilities in 10 projects, evaluating it in real-world scenarios to confirm its practicality.

To the best of our knowledge, this is the first work to apply Micro-segmentation (MSG), which is derived from zero trust principles, to the discovery of attack surfaces and the refinement of data-level permissions in RESTful APIs. Among the 94 RESTful APIs analyzed in the selected GitHub projects, BolaZ achieved recall rates of 97% for attack injection points and 87% for authorization intervals, with only one false positive. The experimental results are promising; BolaZ successfully defended against known BOLA vulnerabilities in our benchmark and identified 35 new BOLA vulnerabilities in 10 real-world projects.

This paper makes the following contributions:

- We propose a novel approach for **identifying BOLA attack injection points** by tracking the data flow of resource IDs. This method enhances detection by analyzing how resource IDs are used throughout the system.
- Our work is the first to apply static taint tracking technology to **infer authorization intervals (MSG intervals)** for BOLA attack injection points. This technique improves the granularity of access control by defining minimum security boundaries based on data flow.
- We introduce a method to **isolate attack injection points to the system's minimum security boundary**, leveraging the resource ID propagation mode. BolaZ is the **first authorization-guided** method that adapts defense rules based on the system's best-practice authorization logic.

The rest of this paper is organized as follows. Section 2 introduces background knowledge on RESTful APIs, BOLA attacks and Micro-Segmentation (MSG). Section 3 demonstrates how the problem of BOLA vulnerability detection is formulated at the methodological level. In Section 4, the technical details of the proposed novel approach, BolaZ, are described. Section 5 shows the details of the implementation. Section 6 describes the experiments and discusses the results. Section 7 provides discussion and future work. Relevant literature is summarized in Section 8. Section 9 concludes this work.

[1]Code availability: https://github.com/wuanbin/bolaz

Just Accepted

2 Background

2.1 RESTful APIs

RESTful APIs are APIs that follow the style of REST architecture [16]. RESTful APIs provide a unified interface for creating (C), reading (R), updating (U), and deleting (D) resources [10]. RESTful APIs are a resource-oriented architecture. Users can identify resources through the URI and correspond to the CRUD operation of the resource through POST, GET, UPDATE, DELETE and PATCH.

2.2 BOLA Attack

BOLA is the number one vulnerability in OWASP API Security [34]. Let's take a concrete example to explain what the BOLA attack is. As shown in Fig.2a, users using order details API (*GET /api/order/{orderid}*) can only read order resources that they have created. Normal users (userA, userB) access the *order* resource using their own created *orderid* (**733,845**). Still, the attacker initiates access to userB's order resources by modifying the *orderid* (***resource ID***) parameter, which is the **injection point** of the BOLA attack. If the developer does not perform an effective resource ID permission check on the API, attackers can gain unauthorized access to other users' sensitive resources. The BOLA vulnerability opens the door for attackers to directly access resources, allowing them to **bypass intended application workflows** and gain unauthorized access to sensitive data.

2.3 Micro-Segmentation (MSG)

MSG [23] follows the core principles of zero trust-minimum permissions, all users are untrusted, and user access to any resource needs to be authenticated. MSG divides resources into several small resource intervals based on service logic in a software-defined way, logically separating resources and restricting the movement of users within resources, enabling the most fine-grained access control of resources. As shown in Fig.2b, according to the system logic, MSG divides the entire *order* resource into small partitions according to user identity. MSG policies allow users to access only the *orderid* created by themselves, and prevents the attacker's horizontal unauthorized access to other users' *order* resources.

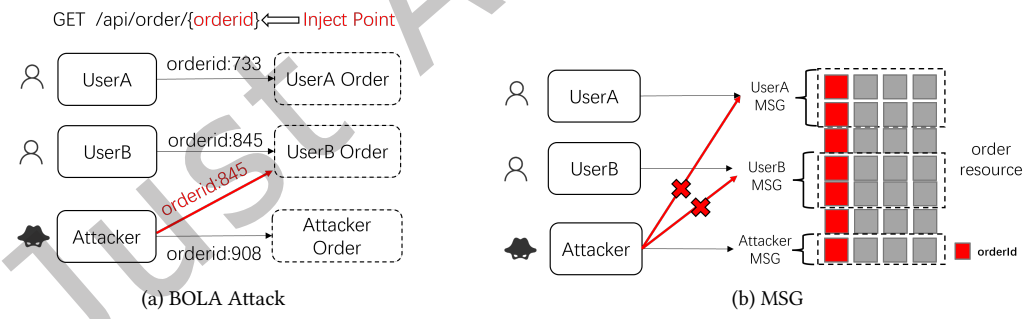


Fig. 2. A BOLA attack on an order details API and its prevention by Micro-Segmentation (MSG)

3 Problem Formulation

3.1 Threat Model

3.1.1 Attacker's Capability. To effectively execute a Broken Object Level Authorization (BOLA) attack, attackers must obtain resource IDs they are not authorized to access. These IDs can be exposed through several API

Just Accepted

vulnerabilities, as shown in Fig.3. The primary vulnerabilities leading to BOLA attacks are Broken Object Property Level Authorization (BOPLA), Unrestricted Access to Sensitive Business Flows (UASBF), and Broken Function Level Authorization (BFLA) [34].

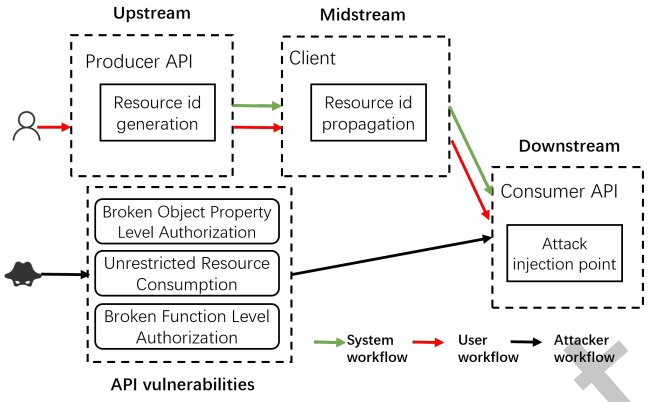


Fig. 3. System, user, and attacker workflows

3.1.2 Vulnerability Techniques.

- **BOPLA**, ranked 3rd in the OWASP API Security Top 10, occurs when APIs unintentionally expose sensitive resource IDs in their responses. Attackers exploit this exposure by extracting these IDs from the API responses and using them to gain unauthorized access to resources.
- **UASBF**, ranked 4th, allows attackers to bypass restrictions and access APIs without proper authentication. When resource IDs are predictable or sequential, attackers can use automated scripts to enumerate these IDs, leading to potential BOLA attacks as they gain unauthorized access to various resources.
- **BFLA**, listed 5th, allows attackers to access APIs with elevated permissions without proper authorization checks. If an API fails to enforce authorization controls, attackers may exploit this to retrieve sensitive resource IDs belonging to other users.

3.2 Terminology

We classify APIs according to the production and consumption of resource IDs, as shown in Fig.4.

- ***Producer API (P-API).*** P-APIs can return the resource ID.
- ***Consumer API (C-API).*** C-APIs consume the resource ID through parameters.
- ***False Producer API (FP-API).*** FP-APIs consume and return the same resource ID, which is essentially C-API.
- ***Producer and consumer API (PC-API).*** PC-APIs consume some resource IDs and produce other types of resource IDs.
- ***Non-producer and non-consumer API (NPC-API).*** NPC-APIs neither produce nor consume resource IDs.

3.3 Problem Definition

Defending against BOLA attacks fundamentally involves confining the resource ID accessed by attackers within the smallest necessary permission scope. As illustrated in Fig.3, the resource ID (attack injection point) is not arbitrarily provided by the user; it is generated upstream in the system workflow and then propagated downstream. Then the resource ID generated upstream of the data stream is the accessible interval of the resource ID of the

Just Accepted

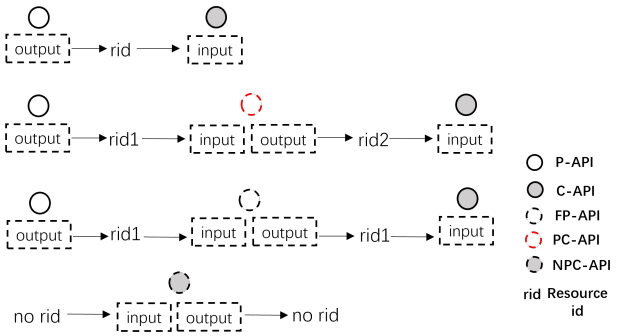


Fig. 4. Classification of APIs based on rid production and consumption

downstream consumer API. Consequently, from the perspective of the resource ID's data flow, the problem of defending against BOLA attacks can be decomposed into three sub-problems.

Problem 1: P-API identification and discovery of resource ID generation rules. The P-API is located upstream in the resource ID data stream, which generates a set of resource IDs according to the system's logical constraints. Therefore, the first sub-problem is to identify the P-API and extract the rules governing resource ID generation.

Formulation: Given an API P, determine whether it produces the resource ID *rid* and the logic rules R of creating the resource ID.

$$P \in API, Rule = \{R_1, ..., R_n\}, R_i = \{rid_1, ...rid_n\} \tag{1}$$

Problem 2: C-API identification and discovery of BOLA attack injection points. The C-API is downstream in the resource ID data stream, utilizing the resource ID propagated by the P-API to perform resource operations. Therefore, the second sub-problem is to identify the C-API and find the BOLA attack injection points in the C-API.

Formulation: Given an API C, determine whether it consumes the resource ID *rid* and which parameters *parm* are associated with the resource ID.

$$C \in API, parm_i \in C, C_{relation} = \{(parm_i, rid_i)\} \tag{2}$$

Problem 3: Data flow association between P-API and C-API. P-API and C-API serve as the resource ID data flow's starting and ending nodes. Analyzing the data flow relationship between P-API and C-API can determine the authorization interval of the BOLA attack injection points.

Formulation: Given a P-API, P, and a C-API, C, determine whether there is a data flow context between P and C.

$$P \in P - API, C \in C - API, API_{relation} = (P, C) \tag{3}$$

3.4 Problem Scope

There are many ways to store and extract resources in RESTful APIs. We define the problem scope to reduce the divergence of the research process and propose an extensible method based on this.

3.4.1 Resource Storage Based on Database. There are many forms of resource storage in RESTful APIs, such as databases, Hadoop, and Spark. The database is the main carrier for resource storage. Therefore, BolaZ considers the database as a resource storage carrier for BOLA attack detection, the same as BOLARAY [18].

Just Accepted

3.4.2 MSG Intervals. BolaZ uses the database as the resource storage carrier, so SELECT statements are the core way for P-API to obtain resource IDs, and SELECT statements support most of the data filtering functions. Server-side codes rarely filter resource IDs received by SELECT statements. Therefore, BolaZ uses SELECT statements of P-APIs as MSG intervals of resource IDs. The ***primary and foreign keys*** obtained by SELECT statements are ***resource IDs***.

MSG intervals are not only determined by SELECT statements because there are two ways for users to delete and update resources: 1). **Server logic control**. P-APIs generate resources that users have permission to delete and modify, and users can delete and modify all resources returned by P-APIs on the client. 2). **Client logic control**. P-APIs return all resources to clients, and front-end codes determine whether current users have permission to use the resource ID to delete or modify the resource.

BolaZ can also be applied to other resource ID generation methods (e.g., INSERT, UPDATE, ORM-based generation, cache). No matter how the generation of resource IDs in P-API changes, BolaZ can provide legitimate MSG intervals for downstream C-API as long as it identifies the generation logic of resource IDs. For example, when the P-API generates the resource ID from the INSERT statement, the MSG interval corresponds to the data previously inserted by the current user.

3.4.3 Propagation invariance of resource IDs. To demonstrate the invariance of the resource ID, we search SpringBoot projects on GitHub and sort them according to the number of stars, giving priority to projects with more attention. The selected projects cover large, medium, and small scales. Some projects also have commercial versions and cover a variety of business application scenarios. The standard for the change of resource ID in the propagation process is that the system passes the resource ID produced by P-API to C-API after modification. We deploy the selected system locally, taking the CRUD operations of each resource as the test unit, and use the Fiddler [43] tool to capture API access logs in real time. The analysis log can accurately determine whether the resource ID has changed in production and consumption.

By analyzing nearly a hundred open-source projects, we find that the resource ID, as the unique identifier of the resource, has assignment and replication operations during the propagation process, and there are few modification and deletion operations (only one case is found in nearly 1,000 APIs). Resource IDs remain invariant over the entire data stream of production, propagation and consumption. Moreover, in API automated testing, the resource ID is also regarded as immutable, e.g., RestTestGen [45] and RESTler [3].

3.4.4 The relationship between the MSG interval and the resource ID invariance. The fundamental principle of BolaZ in detecting BOLA vulnerabilities is based on the MSG interval of user identities, ensuring that users receive only the minimum necessary authorization while blocking unauthorized access by attackers. It is precisely because of the invariance of the resource ID in the propagation process that, as long as we determine the MSG interval generated by the P-API associated with the C-API, BolaZ can determine the legal authorization interval of the attack injection point in the C-API.

4 Approach

In this section, we give an overview of the BolaZ framework and propose three modules to solve our three sub-problems as described in Fig.5.

4.1 Overview

(a) P-API identification and MSG interval generation. ① BolaZ analyzes which SELECT statements generate resource IDs (primary and foreign keys) from server-side source code. ② Resource IDs generated by SELECT statements are propagated in the Mapper layer (database processing layer) and the Service layer (application logic processing layer). ③ Resource IDs are propagated to the return value of the API. BolaZ determines if the

Just Accepted

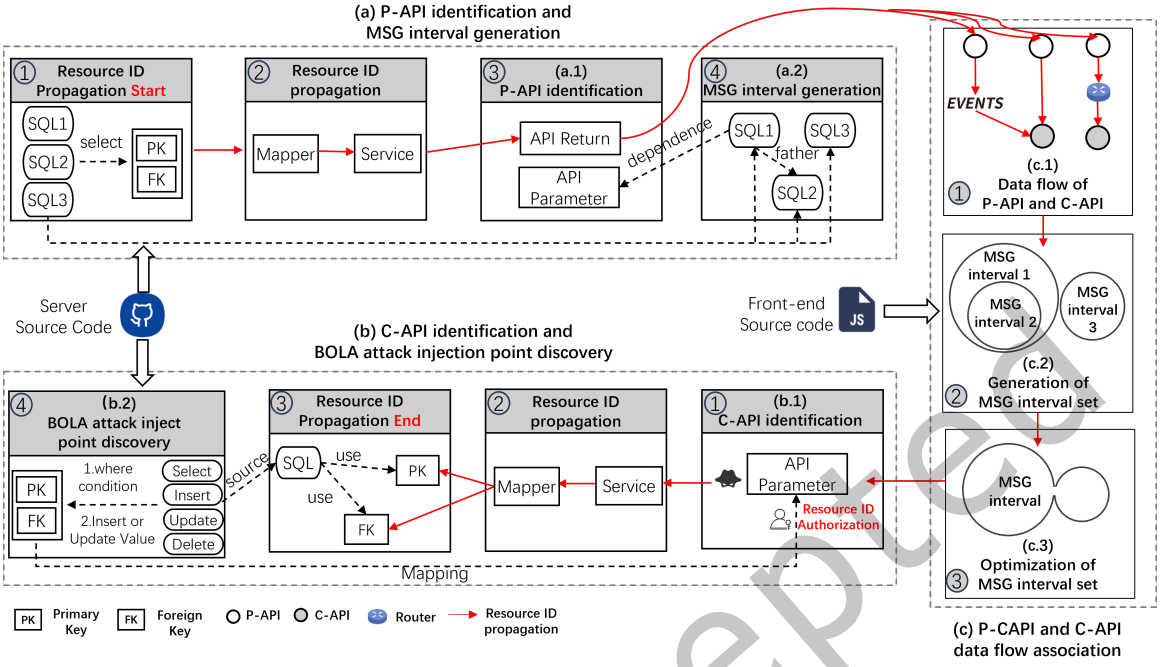


Fig. 5. The overview of BolaZ

API is P-API. ④ BolaZ analyzes the dependencies between P-API parameters and SELECT statements, explores the parent-child relationship between SELECT statements, and generates MSG intervals of resource ID.

(b) *C-API identification and BOLA attack injection point discovery.* ① BolaZ tracks the data flow of API parameters in the server source code. ② API parameters are propagated at the Service and Mapper layers. ③ BolaZ only retains the API parameter data flow propagating to the SQL statement's primary or foreign key (resource ID). ④ BolaZ uses the mapping relationship between API parameters and primary and foreign keys to identify C-API and finds the injection point of the BOLA attack.

(c) *P-API and C-API data flow association.* ① BolaZ explores data flows of resource IDs in front-end code from P-API to C-API and innovatively obtains the relationship between P-API and C-API. ② MSG intervals of multiple P-APIs provide MSG intervals for attack injection points of C-APIs and generate the MSG interval set. ③ BolaZ improves the performance of resource ID authorization checking by optimizing the MSG interval set.

4.2 P-API Identification and MSG Interval Generation

The P-API is the upstream of the resource ID propagation and generates the resource ID's MSG interval. Therefore, BolaZ's first step is to identify the P-API and get the MSG interval of the resource ID.

BolaZ identifies the P-API by checking whether the API generates resource IDs, and also extracts the generation rule (the MSG interval) for the current user's resource ID within the API. To extract this MSG interval, BolaZ traces the resource ID from the SQL layer to the API's return value. For generality, the identification of P-APIs and the extraction of MSG intervals are not restricted to SELECT statements. Although we focus on SELECT as the main carrier for generating MSG intervals, the tracking and extraction approach of BolaZ can be migrated to other carriers as well. As shown in Fig.6, the three SELECT statements produce MSG intervals for the resource

Just Accepted

ID(*blog_id*) of *blog*, respectively. The *MSG interval a* contains the *blog_id* created by user A himself. The *MSG interval b* contains the *blog_id* created by user b himself. The *MSG interval c* contains all *blog_id*. The false MSG interval only reads the amount of data in the *blog* table and does not generate resource ID.

As shown in Equation 4, the MSG interval is the set of resource IDs that P-API has permission to obtain under the current user identity. P represents P-API, u represents current user, R represents resource, $\Phi_P(rid, u)$ represents resource IDs' authorization rules extracted from P-API.

$$MSG_{P,u}(R) = \{rid \mid rid \in \mathcal{R},\ \Phi_P(rid, u) = \text{true}\} \tag{4}$$

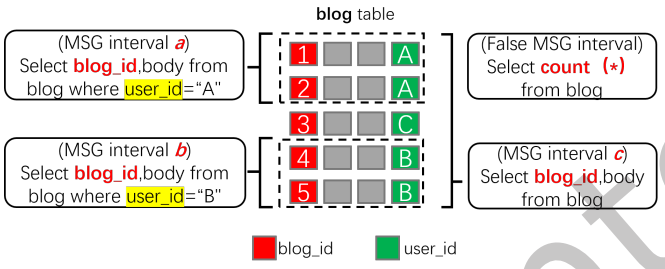


Fig. 6. MSG intervals

4.2.1 P-API Identification. Usually, the GET type API returns the resource ID [48]. BolaZ takes the GET type API as the starting point and uses the taint tracking technology to obtain SELECT statements associated with the return value in the API.

As shown in Fig.8a, BolaZ tracks the propagation of tokens in the API layer and SQL layer, and obtains the SELECT statements associated with the API return value. By analyzing the Abstract syntax tree (AST) of the SELECT statement, BolaZ determines that the SELECT statement obtains the primary key *id* of the *blog* table. Only when the associated SELECT statement obtains the resource ID columns of the table, BolaZ determines the API as P-API. However, the API that produces the resource ID is not necessarily P-API. As shown in Fig.7, SQL-(a) produces the same *blog ID* as the API parameter, not creating a new *blog ID*. BolaZ considers the API for creating and consuming the same resource ID as an FP-API, not a P-API.

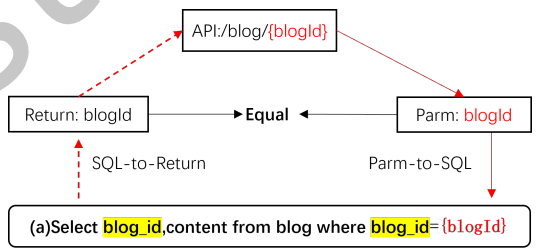


Fig. 7. FP-API Identification.

Listing 1 Reduction of MSG intervals

```
1: Select a_id,addr from address where user_id="uid" and detail="userinput"
     limit start,stop
```

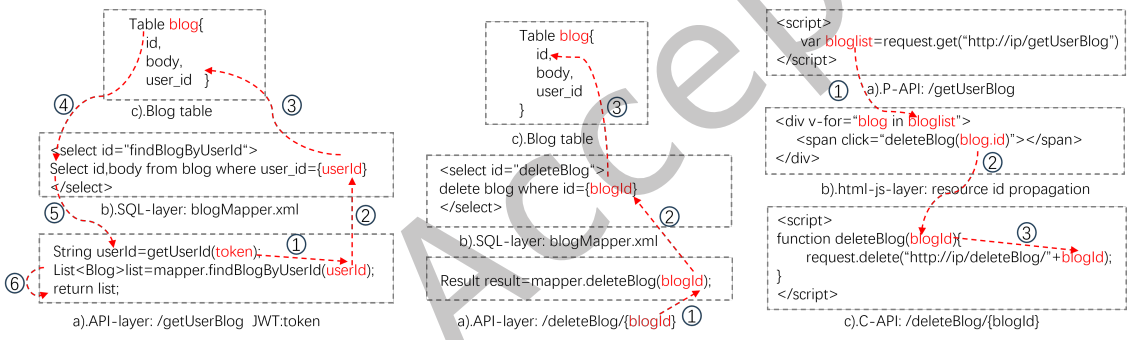
Just Accepted

```
2: Select a_id,addr from address where user_id="uid"
```

4.2.2 MSG Interval Generation. Once the P-API is identified, the SELECT statements associated with the P-API return value are the MSG intervals. MSG intervals must maintain integrity to isolate resource IDs to the maximum authorization range. Additionally, since MSG intervals generated by P-APIs may have dependency conditions, BolaZ extracts these dependencies to ensure the accuracy of the authorization range.

Integrity of the MSG interval. SELECT statements represent MSG intervals. Still, there may be some conditions in SELECT statements to reduce the range of MSG intervals. As shown in Listing 1(1), the SQL statement obtains the user's address. "Detail" condition corresponds to the keyword search function of the address, and the user can make the detail condition cover the entire table without entering the keyword. "Limit" corresponds to the address paging function, and users can also obtain all data through the page turning function. Conditions of this type will reduce MSG intervals. *User_id* is the foreign key of the *addr* table, which is a necessary condition to determine MSG intervals. Therefore, BolaZ preserves only the foreign key and non-user input conditions in SELECT statements, as shown in Listing 1(2).

Dependence of the MSG interval. The complete MSG interval preserves the conditions of foreign keys, but these conditions are not necessarily independent. BolaZ obtains the dependency conditions of foreign keys from external inputs (API parameters and tokens) and internal associations (parent resource IDs) of API.



(a) P-API identification (b) C-API identification (c) P-API and C-API data flow association

Fig. 8. Resource ID production, propagation and consumption

First, BolaZ analyzes the API parameters. The parameters of the P-API are propagated to the foreign key in the MSG interval, indicating that the P-API first consumes a specific type of resource ID and then produces another kind of resource ID. As shown in Fig.9, the *comment ID*'s MSG interval depends on the *blog ID*'s MSG interval. BolaZ tracks the propagation data flow of parameters to find the mapping relationship between parameters and foreign keys.

Second, the API token represents the encrypted information of the user's own identity. Under the premise that the token is not leaked, the attacker is not capable of launching an attack by modifying the token. Therefore, tokens are not considered the potential injection points of BOLA attacks. However, after propagation, the token will be converted to the user ID, which may be passed to the conditions of SELECT statements in P-API. It is also a dependency condition for generating MSG intervals. BolaZ tracks the token propagation to discover whether tokens are associated with userId.

Third, there may be a parent-child dependency between resource IDs in P-APIs. Obtaining the MSG interval of the parent resource ID is the premise of calculating the MSG interval of the sub-resource ID. As shown in Fig.10,

Just Accepted

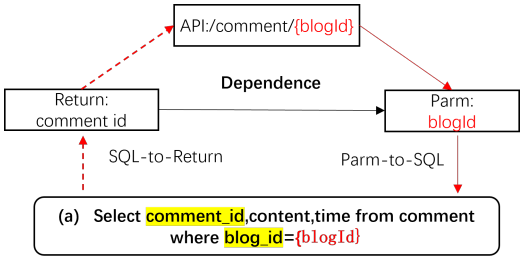


Fig. 9. PC-API identification. the API generates *comment IDs* while consuming the *blog ID*, which is both a C-API and a P-API.

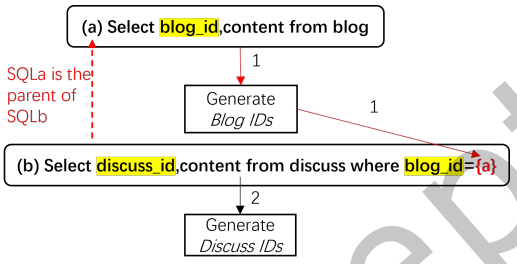


Fig. 10. Dependency on resource ID

the API uses SQL-(a) and SQL-(b) to produce MSG intervals of *blog ID* and *discuss ID*, respectively. SQL-(a) for SQL-(b), and SQL-(b) depends on SQL-(a). BolaZ tracks the dependency of resource IDs within P-API between SQL statements.

Finally, BolaZ identifies the P-API by analyzing the SELECT statement that returns the resource ID in the API and extracts MSG intervals.

4.3 C-API Identification and BOLA Attack Injection Point Discovery

C-API is downstream from the propagation data flow of resource ID. The BOLA attack takes the resource ID parameter of the C-API as the injection point, so BolaZ needs to identify the C-API and determine which parameters have a mapping relationship with the resource ID.

BOLA attack injection points exist in user-controlled resource ID parameters, which eventually propagate to the SQL statement's primary or foreign key. Since the resource ID parameter does not change in the C-API propagation process (detailed in 3.4.3), BolaZ also uses taint tracking technology [24] to analyze which API parameters are propagated to SQL statements, and determines which resource ID is associated with API parameters by analyzing the AST of SQL statements.

Corradini et al.[10] used common naming practices to identify the resource ID parameters in the API. However, this identification method relies on naming rules and is prone to false negatives and false positives. BolaZ does not require heuristic methods and rule libraries to determine the injection points of BOLA attacks. Even if the same resource ID has multiple parameter names, BolaZ can still identify the resource ID parameters in the C-API through the mapping relationship between the API parameters and the primary and foreign keys of the SQL statement.

Just Accepted

As shown in Fig.8b, BolaZ uses API parameters (*blogId*) as taint sources and SQL statements as taint propagation endpoints. BolaZ found that *blogId* eventually spread to the *id* condition of Delete statement to delete *blog*. BolaZ determines that the *deleteBlog{blogId}* is C-API, and the *blogId* parameter consumes the primary key *id* of the *blog* table.

BolaZ determines whether the API parameters consume the resource ID from the context correlation between API parameters and primary or foreign keys in the four operation types of SQL statements, as shown in Table 1.

Table 1. C-API identification

Type	**Mapping**
Select	There is a mapping relationship between API parameters and the where condition's primary key or foreign key in the SQL statement.
Delete	There is a mapping relationship between API parameters and the where condition's primary key or foreign key in the SQL statement.
Insert	API parameters are mapped with the primary or foreign key of the inserted value in the SQL statement.
$Update_1$	There is a mapping relationship between API parameters and update values in SQL statements.
$Update_2$	There is a mapping relationship between API parameters and the where condition's primary key or foreign key in the SQL statement.

4.4 P-API and C-API Data Flow Association

Effective defense against BOLA attacks hinges on capturing API contexts [42]. In system development, developers retain the context relationship between P-APIs and C-APIs when writing front-end code. Therefore, BolaZ can also analyze the propagation relationship of resource IDs between P-APIs and C-APIs in front-end code through taint tracking technology.

As shown in Fig.8c. When the page is loaded, the Javascript code calls the P-API (*/getUserBlog*) to get the *blog* created by the user himself. The HTML code renders the *blog* resources returned by the P-API and provides a *click* event so that the user can call the *deleteBlog* function. When the user initiates a click event in the page, the Javascript code calls the C-API (*/deleteBlog/{blogId}*) to delete the corresponding *blog* resource. BolaZ takes P-API as the starting point of taint tracking and C-API as the end point of taint tracking determines which P-API provides resource IDs for the C-API.

4.4.1 Identify The Data Flow of P-APIs And C-APIs. BolaZ captures the propagation of resource ID within and between HTML pages. First, we analyze the propagation of resource ID within the HTML page. As shown in Fig.11, BolaZ divides the internal propagation of the page into four data flow patterns, with P-API as the starting point of data flow propagation. C-API will terminate the data flow after consuming the resource ID. The router is used to realize the function of jumping from one page to another, and the parameters can be carried out during the jumping process.

- *P-API to C-API*: P-API propagates resource IDs directly to C-API
- *P-API to Event to C-API*: P-API propagates resource IDs to C-API through HTML events.
- *P-API to Router*: P-API propagates resource IDs to Router.
- *P-API to Event to Router*: P-API propagates resource IDs to Router through HTML events.

Secondly, the propagation of resource ID between HTML pages can complement the data flow of P-API and C-API between multiple HTML pages. As shown in Fig.11, BolaZ uses two data flow patterns to receive the resource ID passed from the parent HTML page.

Just Accepted

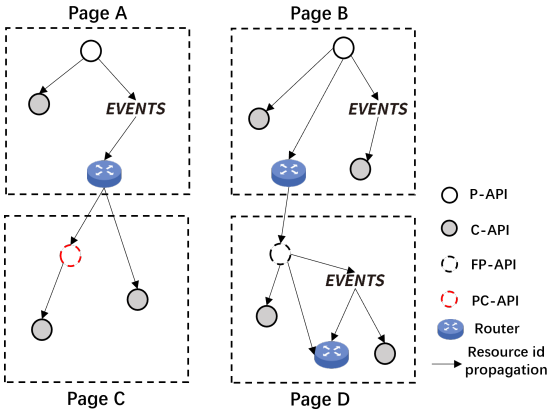


Fig. 11. Data flow between P-API and C-API

- *Router to PC-API*: Router receives resource IDs of parent pages and propagates them to PC-API.
- *Router to C-API*: Router receives the resource ID of parent pages and propagates them to C-API.

Through the above six data flow patterns, BolaZ tracking API workflows to obtain the logical relationship between attack injection points and MSG intervals, and ensuring that the authorization policy of injection points constantly changes with API workflows.

4.4.2 Generation of MSG Interval Set. The MSG interval of the resource ID produced by the P-API is the authorization interval of the associated C-API attack injection point. However, the three characteristics of MSG intervals generated by P-APIs lead to the limitations of the authorization interval of attack injection points in C-APIs: the diversity of resource IDs, the dependence between resource IDs, and the incompleteness of foreign key resource IDs. Therefore, BolaZ performs the following three operations on MSG intervals.

Matching. P-API may create MSG intervals for multiple resource IDs. Therefore, BolaZ must match the resource ID consumed by C-API with the resource ID created by P-API. As shown in lines 2-7 of Algorithm 1, BolaZ matches the MSG interval of attack injection points according to the data flow of P-API and C-API.

Dependence Resolving. When there is any dependency between resource IDs, BolaZ will analyze the MSG interval of the parent resource ID. If the MSG interval of the parent resource ID covers all the data in the database table, the child resource ID will discard the limitation of the parent resource ID. Otherwise, BolaZ will update the condition of the parent resource ID to the MSG interval of the parent resource ID. As shown in lines 8-9 of Algorithm 1, BolaZ obtained the dependency relationships between resource IDs and generated MSG intervals for child resource IDs based on those for the parent resource IDs.

Backtracking. The primary or foreign key obtained by P-API may become the resource ID that C-API requires. The MSG interval of the primary key is complete, which can provide the C-API with a full range of permissions for the available resource ID. The foreign key only has a partial mapping relationship with the primary key of other resources, and the generated MSG interval is incomplete.

The foreign key is inserted into the database by the C-API of the resource creation type (POST), so the MSG range of the foreign key is created by the P-API that provides the resource ID for the POST C-API. As shown in Fig.12, in data stream 1, the P-API of Resource B passes the resource ID of the primary key to the POST C-API of Resource A, and the C-API stores the primary key of Resource B as the foreign key of Resource A. In data stream 2, the P-API propagates the resource ID of the foreign key to the C-API. The MSG interval of the foreign

Just Accepted

Algorithm 1 MSG Construction: P-C API Association & Foreign-Key Backtracking

Require: $\mathcal{P}$: Producer APIs (P-API), $\mathcal{C}$: Consumer APIs (C-API), $SQL(P)$: SQL queries of P-API, U: User identity
Ensure: $\mathbb{M}$: MSG intervals, $\mathcal{M}_{PC}$: P-C API association set, $\mathcal{D}$: Resource dependency graph, $\mathbb{M}^*$: Refined MSG intervals after backtracking
Auxiliary Functions & Notation
- $\mathcal{K}$: Intrinsic authorization constraints
- Clean(Q): Clean SQL; keep U, **fk**, $\mathcal{K}$; remove filters/LIMIT
- FlowOf(P, C, rid): Establish $P \rightarrow C$ via rid flow
- DependOn(r_p, r_c): r_c depends on r_p; add edge to $\mathcal{D}$
- Based($M(r_p), M(r_c)$): $M(r_c)$ based on $M(r_p)$
- TraceToSource(**fk**): Trace **fk** to its PK-generating P-API
- $\mathbb{M}(P)$: Get MSG interval generated by P-API P
- $\mathbb{M}(C)$: Get MSG interval associated with C-API C

1: $\mathbb{M} \leftarrow \emptyset,\ \mathcal{M}_{PC} \leftarrow \emptyset,\ \mathcal{D} \leftarrow \emptyset$
Part 1: Build MSG & Associate P-C APIs
2: **for all** $P \in \mathcal{P}$ **do**
3: **for all** $Q \in SQL(P)$ returning rid **do**
4: $Q^* \leftarrow \text{Clean}(Q)$
5: $M_P \leftarrow \{ rid \mid Q^*(rid, U, \mathcal{K}) \}$
6: $\mathbb{M} \leftarrow \mathbb{M} \cup \{(P, M_P)\}$
7: $\mathcal{M}_{PC} \leftarrow \mathcal{M}_{PC} \cup \{ (P, C) \mid \text{FlowOf}(P, C, rid) \}$
8: $\mathcal{D} \leftarrow \mathcal{D} \cup \{ (r_p, r_c) \mid \text{DependOn}(r_p, r_c) \}$
9: $M(r_c) \leftarrow \text{Based}(M(r_p), M(r_c))$
10: **end for**
11: **end for**
Part 2: Foreign-Key Interval Backtracking
12: **for all** $(P, C) \in \mathcal{M}_{PC}$ **do**
13: $M_C \leftarrow \mathbb{M}(C)$
14: **if** M_C is defined on foreign key **fk then**
15: $P_{\text{src}} \leftarrow \text{TraceToSource}(\textbf{fk})$
16: $M_{\text{pk}} \leftarrow \mathbb{M}(P_{\text{src}})$
17: $\mathbb{M}^*(C) \leftarrow M_{\text{pk}}$
18: **else**
19: $\mathbb{M}^*(C) \leftarrow M_C$
20: **end if**
21: **end for**
22: **return** $\mathbb{M},\ \mathcal{M}_{PC},\ \mathcal{D},\ \mathbb{M}^*$

Just Accepted

key resource ID in data stream 2 comes from the P-API in data stream 1. Therefore, BolaZ converts the MSG interval of the foreign key into the MSG interval of the primary key of the P-API associated with the POST C-API through data stream backtracking, as shown in data stream 3. As shown in lines 12-21 of Algorithm 1, BolaZ traces the MSG interval of the primary key corresponding to the foreign key.

C-API may have multiple associated P-APIs, which means it is related to numerous MSG intervals, thus forming a set of MSG intervals.

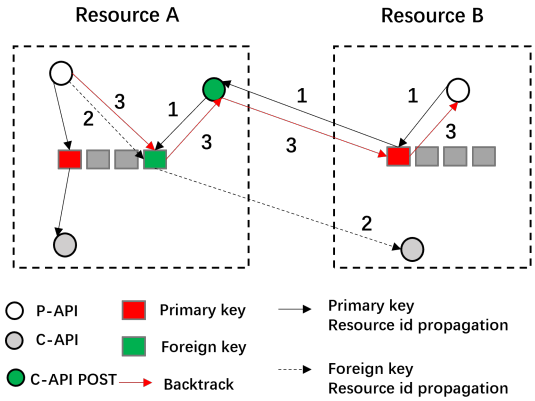


Fig. 12. Data flow backtracking

4.4.3 Optimization of MSG Interval Set. There may be subset or intersection relationships between the internal elements of the MSG interval set, and performing repeated interval queries will result in considerable performance costs. Therefore, BolaZ defines the merging rules of SELECT statements to reduce its number under the premise of ensuring that the MSG interval does not change, thus reducing the number of SELECT statements and the query of repeated intervals.

First, we define the identifiers in the merging rules, as shown in Table 2. Then, we discuss the merging rules of SQL statements from the perspective of the coincidence relationship between MSG intervals, As shown in Fig.13.

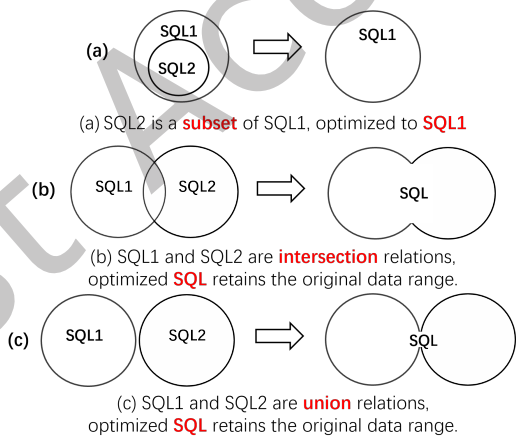


Fig. 13. MSG interval optimization

Rule#1:Subset rule. When there is a subset relationship between the two SQL statements, the BolaZ merges the two SQL statements into a parent SQL statement. As shown in Listing 2, the MSG interval of addressID represented by SQL2 is a subset of SQL1. BolaZ takes SQL1 as the MSG interval of addressID.

Precondition:

- $S_i \in S_q$, $S_j \in S_q$, $T_i = T_j$

Just Accepted

Table 2. Merge rule identifier

Type	Meaning
S_q	Single table query.
M_q	Multi table query.
M_t	The main table in multi-table query.
S_i	The ith MSG interval (SQL statement) associated with the C-API.
T_i	T_i is the table corresponding to resource ID produced in S_i.
W_i	W_i is the set of where conditions related to T_i in S_i.

- $S_i \in M_q$, $S_j \in M_q$, $T_i \in M_t$, $T_j \in M_t$, $T_i = T_j$
- $S_i \in S_q$, $S_j \in M_q$, $T_j \in M_t$, $T_i = T_j$

if:

$W_i = \emptyset$ or $W_i \subseteq W_j$

then:

$S_i \cup S_j = S_i$

Listing 2 Subset rule example

```
1: Select a_id,addr from address
2: Select a_id,addr from address where user_id="uid"
```

Rule#2:Intersection and union rule. When two SQL statements have an intersection or union relationship, the BolaZ merges the where conditions of the two SQL statements. As shown in Listing 3, the MSG intervals for addressID represented by SQL1 and SQL2 do not intersect, but both are associated with the MSG intervals of the C-API. To ensure the integrity of the authorization detection, BolaZ combines SQL1 and SQL2 as the MSG intervals of addressID.

Precondition: $S_i \in S_q$, $S_j \in S_q$, $T_i = T_j$

if:

$W_i \cap W_j = \emptyset$ or $W_i \cap W_j \neq \emptyset$, $W_i \not\subseteq W_j$

then:

$S_i \cup S_j = W_i \cup W_j$

Listing 3 Intersection and union rule example

```
1: Select a_id,addr from address where status=0
2: Select a_id,addr from address where status=1
```

BolaZ uses the optimized MSG interval to perform a resource ID authorization check on the BOLA attack injection point of the C-API. Suppose the optimized MSG interval can cover all the database table data. In that case, it shows that the resource ID consumed in the C-API is unlimited, and the BolaZ does not perform authorization checks on the C-API.

4.4.4 Justification for Generality of BolaZ. We recognize that the resource IDs (injection points) of the C-API are not arbitrarily set by users; instead, they are generated by the P-API within the resource ID data stream. The principle of tracking the production, propagation, and consumption of resource IDs to determine authorization intervals is general. Therefore, based on the resource ID propagation data stream, BolaZ micro-segregates the injection points into the smallest possible authorization intervals. Regardless of whether the storage method

Just Accepted

changes, the resource ID propagation process changes, or the means of obtaining the resource ID changes, as long as the generation logic, propagation logic, and consumption logic of the resource ID are known, the authorization interval for the BOLA vulnerability can be determined.

5 Implementation

In this section, we detail the implementation of BolaZ, a tool for detecting BOLA vulnerabilities in SpringBoot-based web applications. As shown in Fig.14, BolaZ includes offline analysis and runtime authorization.

Based on CodeQL's analysis framework for Java and JavaScript libraries, we have implemented BolaZ. CodeQL is a highly customizable taint-tracking analysis tool that provides global and precise data flow analysis capabilities by converting code into a semantic database, allowing developers to customize methods to reduce interruptions in taint propagation, and supporting data flow analysis for the SpringBoot framework. These advantages are sufficient for CodeQL to provide adequate precision for inferring MSG intervals.

CodeQL does not have any existing libraries to support the automated generation and propagation of resource IDs. In contrast, CodeQL is prone to breakdowns at the framework level, aliases, and API associations that cause resource ID tracking to fail. We developed customized CodeQL data flow rules to obtain the complete data flow of resource ID propagation. The core contribution of BolaZ lies in inferring MSG authorization intervals of attack injection points based on the data stream of resource IDs, rather than the taint tracking engine itself.

Due to the complexity of control flow, front-end logic, and coding patterns in the real world, it is impossible for BolaZ to completely reliably capture the data flow that transmits resource IDs. BolaZ 's static taint analysis is explicitly designed as a best-effort analysis rather than a fully sound (complete) one.

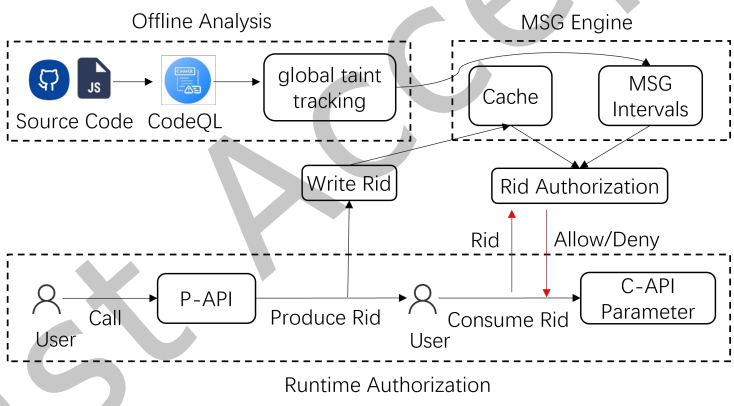


Fig. 14. Implementation of the BolaZ

5.1 Offline Analysis

BolaZ utilizes CodeQL's taint tracking tool [8] to trace the data flow of resource IDs. BolaZ focuses on determining MSG intervals for APIs that operate under ordinary user permissions. If an attacker gains administrator privileges, he has permission to operate on all resources and does not need to launch BOLA attacks. Not performing authorization checks on administrator rights APIs also reduces the performance overhead of BolaZ.

The first step is identifying P-APIs. BolaZ uses global taint tracking [9], treating API parameters or function expressions as the source and API returns as the sink. During the data propagation process, BolaZ analyzes the abstract syntax tree (AST) of SQL statements to extract resource IDs and determine the MSG intervals produced

Just Accepted

by the API. The generated MSG intervals are stored in the MSG engine. BolaZ also tracks token propagation. While RESTful APIs typically use token-based authentication methods such as JWT [22] or OAuth [15], the specific implementation varies across applications, making it challenging to extract token flags from source code. Therefore, BolaZ requires developers to provide the relevant token code flags.

CodeQL can only track the calling methods from resource IDs to the Mapper layer, but it cannot pinpoint the specific SQL statements. BolaZ links the methods of the Mapper layer with the SQL statements, as well as the corresponding relationships between the method parameters and the SQL statement parameters, and obtains the complete SQL statements in the MSG interval. Additionally, if there are associations among multiple resource IDs generated by the API, CodeQL is interrupted when tracking the associations between resource IDs, resulting in the failure to identify the MSG intervals of sub-resource IDs. As shown in Listing 4, the MSG interval of tagID depends on blogID, and CodeQL experiences an interruption between the *searchBlog* and *findTagByBlogId* methods at the SQL layer.

Listing 4 Framework abstraction interrupt instance

```
1: List<Blog> blogs = blogDao.searchBlog();
2: for(Blog blog:blogs){
3:     blog.setTags(tagDao.findTagByBlogId(blog.getId()));
4: }
```

The second step is the identification of C-API. We take the API parameters and variables within the functions as the source for CodeQL, and the SQL in the Mapper layer as the sink. Subsequently, by analyzing the mapping relationship between API parameters and primary keys/foreign keys, we determine the C-API and injection points. Similarly, in the Mapper layer, the data flow tracking in CodeQL is interrupted. We capture the complete propagation path of the resource ID based on the parameters, SQL statements, and their corresponding relationships.

Finally, in the context of the association between P-API and C-API. Based on the six propagation modes mentioned earlier, we obtain the complete path of resource ID propagation by using P-API/router/event as the source and C-API/PC-API/event as the sink. CodeQL can track the propagation of resource IDs from the P-API to the C-API and from the router to the C-API through custom extension rules. However, the propagation of resource IDs from P-API to event, from P-API to router, and from event to router is interrupted. This is because CodeQL does not support tracking the propagation path from JavaScript to HTML. Therefore, BolaZ utilized CodeQL to track the variables that the resource IDs produced by P-API reached, and by analyzing the HTML AST, tracked which routers and events the resource IDs obtained from P-API propagated to, thereby completing the data flow of resource ID propagation on the client side.

During the taint tracking process of resource IDs, alias/field propagation (e.g., set, copyList) can be interrupted. BolaZ has added AdditionalTaintStep rules for some common interruption methods to enhance the coverage of CodeQL and reduce false negatives.

5.2 Runtime Authorization

BolaZ checks the authorization of resource ID according to MSG intervals generated by offline analysis when the system is running. Resource IDs used by users must exist in MSG intervals, otherwise, the C-API call will be blocked. We consider that when the database data size is large, if database queries are performed at each C-API authorization check, it will lead to high latency. C-APIs will be called in a short time after users call P-APIs. Hence, the MSG engine caches resource IDs produced by P-APIs. If the cache is not hit, a database query is performed. A cache or database hit means the C-API has access to the resource ID.

Just Accepted

6 Evaluation

6.1 Research Questions

RQ1: How is the effectiveness of BolaZ identifying API types and associations?
First, BolaZ transforms the defense of BOLA attacks into three sub-problems in the resource ID workflow(see 3.3). Hence, our experiments first evaluate the effectiveness of BolaZ by verifying the technical consistency of each 'module' (see 4.1) through data flow analysis.
RQ2: How much extra performance overhead does BolaZ cost?
Second, BolaZ performs authorization checking on resource IDs by caching or database queries, which results in extra performance overhead. We evaluate the performance overhead of BolaZ by comparing the original system and the BolaZ-added system in three aspects: P-API storage cache, C-API cache query and database query.
RQ3: How effective is BolaZ's defense against real-world BOLA attacks?
BolaZ should be able to defend against BOLA attacks in the real world. We first use the CVE vulnerability to evaluate the effectiveness of BolaZ's defense against existing vulnerabilities. Second, we verify BolaZ's ability to detect unknown vulnerabilities by scanning the BOLA vulnerability of open-source projects.
RQ4: How does BolaZ compare with the SOTA approach?
BOLARAY is the most closely relevant work to BolaZ and is the SOTA method for detecting BOLA vulnerabilities. Therefore, we compared the vulnerability detection effect of BOLARAY and BolaZ in real-world projects.
RQ5: How does complementing resource ID propagation at different stages contribute to the success of BolaZ?
During the process of tracking the propagation of resource IDs, CodeQL may encounter interruptions (e.g., framework abstraction, aliasing). Therefore, we use ablation experiments to verify the necessity of BolaZ in completing the data flow of resource ID propagation.

6.2 Dataset

Table 3. Statistics on evaluation projects

Project	Stars	Files	LLOC	APIs	Description
Blog-master [30]	616	106	42,366	57	Blog System
BookStore-master [40]	314	168	29,944	71	Bookstore Project
Mall-master [27]	78.8k	121	15,467	65	E-commerce System
NewbellMall-master [33]	1.4k	140	16,546	62	Mall System
IceCms-master [44]	1.7k	241	80,308	109	Content Management System
MusiceWbsite-master [46]	5.7k	133	42,906	63	Music Website
OnlineExam-master [47]	2k	106	8,116	46	Online Examination System
UniversityForum-master [11]	196	64	13,279	16	University Campus Forum
InformationSystem-master [6]	2k	20	2,344	4	People Information System
OnlineMall-master [41]	32	118	10,640	33	Online Mall
Total	-	1,217	261,916	526	-

6.2.1 How to collect projects. As the implementation of BolaZ is on the SpringBoot framework in Java language, we searched for repositories that depend on SpringBoot on GitHub [39], SourceCodeExamples [38] and LibHunt [25] and filtered them by application types, authentication modes, and logic control modes, as shown in Table 3.

The selected 10 projects cover various application types, two authentication modes, and two logic control modes.

Just Accepted

1). Application type. There are similarities in the business types of open-source projects. The project we selected contains Blog, Bookstore, E-commerce, Content Management System, Online Examination System, People Information System, etc. We screen systems with different business logic so that BolaZ can cover a variety of business scenarios.

2). User authentication mode. User authentication modes include tokens and cookies, so the system we have chosen covers both user authentication modes.

3). Logical control modes (detailed in section 3.4.2).

Based on these three, the selected projects are sufficient and can effectively cover our scope of problems. We focus on the API data flow, which requires a good coverage of business types.

6.2.2 RQ1 & RQ2. There is no tool to achieve the classification and association of API. To construct the dataset's ground truth, we need to deploy projects locally and manually count and identify P-API, C-API and the relationship. The whole process is complex and time-consuming. So we selected five systems from Table 3 based on application types, logical control mode, and user authentication modes to contribute a benchmark to verify the effectiveness of BolaZ, as shown in Table 4. Sufficient to verify the effectiveness of BolaZ.

We manually analyzed the API categories and relationships to build the ground truth data. First, we deployed and ran the project locally, logging in as a regular user to access all functionalities. Simultaneously, we used Fiddler [43] to capture the API list. Next, we examined the API server's source code. Given that the current architecture follows a three-tier model, we were able to quickly categorize the APIs. We then analyzed the front-end source code to trace the propagation paths of resource IDs both within and across pages. There are four people involved in manual annotation, all with rich programming experience, and they have sufficient ability to cover the annotation work of the projects in the dataset. Our team is divided into two groups, analyzing projects independently, followed by cross-validation of the results. Across the five projects, we identified a total of 40 P-APIs and 54 C-APIs. Moreover, the injection point of each C-API and the MSG interval corresponding to the injection point are determined.

6.2.3 RQ3-1. We searched the CVE vulnerability database for BOLA (a.k.a., IDOR) vulnerabilities. The results indicate that only a few are from Java projects (e.g., SpringBoot). Finally, we screened out three vulnerabilities corresponding to the three modes of BOLA attack from the CVE-2023-36100 [13] and CVE-2023-32310 [12].

6.2.4 RQ3-2. The survey of existing work [18], [5], [37], [29] found no current large-scale benchmarks. We detect unpublished BOLA vulnerabilities in these 10 projects to evaluate the effectiveness of BolaZ in the real world.

6.2.5 RQ4. BOLARAY is used to detect BOLA vulnerabilities in PHP. We adapted BOLARAY to the SpringBoot framework and compared it with BolaZ by detecting BOLA vulnerabilities in the collected 10 projects.

6.2.6 RQ5. We conduct experiments on the five projects used by RQ1 to identify BOLA attack injection points and MSG intervals to verify the necessity of BolaZ in completing the resource ID propagation data stream.

6.3 Metrics

In our experiment, we evaluated the following indicators. We use Precision and Recall to measure the effectiveness of API classification and API association. True Positive(TP) represents the number of correctly classified APIs; The number of API pairs that are correctly associated. False Positive (FP) represents the number of APIs classified as P-API (C-API) but not P-API (C-API); The number of API pairs that are judged to be associated with P-APIs and C-APIs but do not exist. False Negative (FN) represents the number of APIs classified as not P-API (C-API) but P-API (C-API); The number of P-API and C-API pairs that are judged not to be associated but are associated.

Just Accepted

Table 4. Benchmark of Identifying API Types and Associations, R denote Relation, LCM denote Logic control mode, UAM denote User authentication mode.

Project	P-API	C-API	API-R	Inject Point	LCM	UAM
NewbellMall	9	14	17	15	Server	Token
OnlineExam	11	20	20	20	Server	UserId
Blog	10	14	49	16	Client	Token
UniversityForum	9	4	10	7	Server	UserId
InformationSystem	1	2	2	2	Server	Token
Total	40	54	98	-	-	-

The formulas for Precision (Pr) and Recall (Re) are as follows.

$$Pr = \frac{TP}{TP + FP}, Re = \frac{TP}{TP + FN} \tag{5}$$

6.4 RQ1: How is the Effectiveness of BolaZ Identifying API Types and Associations?

6.4.1 Setup. The experiment was completed in two steps. First, we use BolaZ to classify the APIs in the project. Suppose there is an error in the API classification. In that case, we will modify the API to the correct type because the premise of accurately associating the API is to obtain the accurate API type. In the second step, we use BolaZ to associate P-API with C-API.

Table 5. Effectiveness of API Classification and Association, P denotes P-API, C denotes C-API, R denotes Relation.

Project	P-Pr	P-Re	C-Pr	C-Re	R-Pr	R-Re
NewbeeMall	8/8	8/9	14/14	14/14	17/17	17/17
OnlineExam	11/11	11/11	20/20	20/20	20/21	18/20
Blog	10/10	10/10	13/13	13/14	42/42	42/49
UniversityForum	9/9	9/9	4/4	4/4	7/7	7/10
InformationSystem	1/1	1/1	2/2	2/2	2/2	2/2

6.4.2 Results. The experimental results of BolaZ are reported in Table 5. BolaZ achieved recall rates of 97% for API classification and 87% for API association, with one false positive. The precision of API classification and association is high because the resource ID, as the unique identification of the resource, is rarely modified in the process of data flow propagation. BolaZ can categorize and associate APIs very accurately without interrupting the taint tracking. We also analyze the reasons for the failure of API classification. By examining the log, we found that CodeQL generated data flow interruption at the *Arrays.asList* method during the taint tracking process, failing P-API identification of the Newbellmall project. The Blog project also failed C-API recognition due to data flow interruption.

We observed that the Online-Exam, UniversityForum and Blog projects have low API association recall rates. First, we analyze Online-Exam and UniversityForum projects. The C-APIs of association failure are all attack injection points of userId, such as *GET /api/score/{current}/{size}/{studentId}*. By analyzing the source code, we found that no P-API provides studentId for C-APIs that fail to correlate, and these studentId are derived from the client's local cookie. The Online-Exam Project saves the *student ID* to the client's cookie after the student user logs in and provides it to the C-API. REST APIs typically use encryption, such as JWT, for authentication, and the Online Exam project's method of using the *user ID* is not secure. At present, the *user ID* will exist in

Just Accepted

more API return data, and it is easy for attackers to use the *user ID* to launch BOLA attacks. Although BolaZ can solve this problem by extending the data flow from client data storage (cookie, local storage) to C-API, this API authentication mode is not secure.

BolaZ generated the only false positive in analyzing API relationships in the online-exam project. After analyzing the source code, we found that the reason was that the client code modified the resource ID generated by P-API and passed it to the downstream C-API, resulting in BolaZ incorrectly associating the context of P-API and C-API. BolaZ needs to analyze the source code to obtain the modification logic of the resource ID to solve this type of false positive. The problem is transformed into extracting operational semantics from the source code.

Secondly, we analyze that the Blog project is due to the existence of client logic control in the system, which leads to the failure of the API association. For example, *v-if= getStoreName() == name ||getStoreRoles().indexOf("ADMIN") >-1*. This condition means that the user's name stored by the client is equal to the user name of the resource, or the stored permission string contains ADMIN so that the user can see the button to delete the blog and have the right to call the API to delete the blog. However, BolaZ cannot accurately determine the meaning of this condition, failing the association between the deletion API of blog resources and the list API.

Table 6. The validity of BolaZ in identifying attack injection points and MSG intervals, IP denotes inject point, MSG+ represents the increase of data stream from local storages to C-APIs.

Project	IP-Pr	IP-Re	MSG-Pr	MSG-Re	MSG+-Pr	MSG+-Re
NewbeeMall	15/15	15/15	15/15	15/15	15/15	15/15
OnlineExam	20/20	20/20	17/18	17/20	18/19	19/20
Blog	15/15	15/16	13/13	13/16	13/13	13/16
UniversityForum	7/7	7/7	4/4	4/7	7/7	7/7
InformationSystem	2/2	2/2	2/2	2/2	2/2	2/2

BolaZ can extract the injection point of the BOLA attack after determining the C-API, and obtain the injection point's MSG interval after obtaining the P-API associated with the C-API. As shown in Table 6, the precision and recall of identifying the injection point of the BOLA attack reached 100% and 98.75%, respectively, and the precision and recall of identifying the MSG interval of the injection point reached 98.88% and 84.68%, respectively.

By analyzing the MSG interval reasoning results, we found that P-API recognition failure did not necessarily affect MSG interval recognition. In the *NewbeeMall* project, the P-API (GET /shop-cart/settle) was not recognized by BolaZ, but there was no downstream C-API for this P-API. Therefore, it did not affect the accuracy of BolaZ's reasoning for MSG intervals in *NewbeeMall*. Similarly, if multiple P-APIs are associated with one C-API. Even if some P-APIs fail to be identified or some associations between P-APIs and C-APIs fail, as long as the C-API is still associated with the correct MSG intervals through the P-APIs, the MSG intervals of the C-API will still be in line with the system authorization logic.

In the experimental results, we found that the recall of MSG interval reasoning in the *UniversityForum* project only reached 57.14%. Through analysis, we discovered that this was because in the previous step, we did not analyze the data flow from the client data storage to the C-API, which consequently led to the failure of 3 MSG interval identifications. When BolaZ added the identification of this type of data flow, the recall rates of MSG interval reasoning in the *UniversityForum* and *OnlineExam* projects reached 100% and 95%, respectively.

Insight #1: Static analysis combined with dynamic analysis is a worth trying direction, which has the potential to refine **propagation logic** of resource ID.

Just Accepted

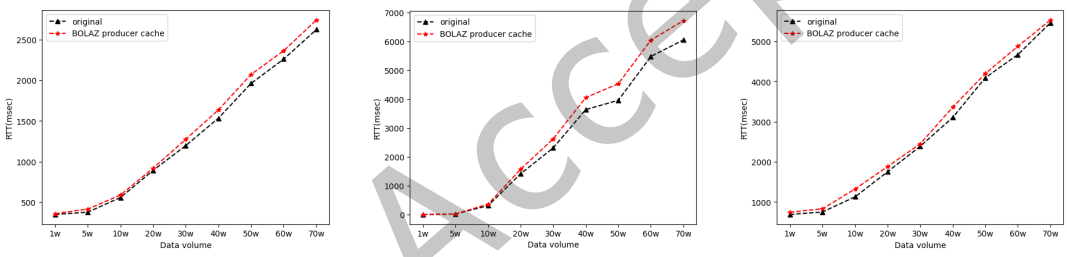
Answer to RQ1: BolaZ achieved recall rates of 97% for API classification and 87% for API association, with few false positives.

6.5 RQ2: How Much Extra Performance Overhead does BolaZ Cost?

6.5.1 Setup. To evaluate the performance overhead caused by BolaZ. We selected three scale projects (small: Blog, medium: NewbeeMall, large: Mall) to verify the performance overhead of BolaZ for the original system. In each project, we selected three pairs of P-API and C-API for performance testing.

We conducted a round-trip latency (RTT) comparison test between the original application system and the BolaZ-added system to evaluate the extra performance overhead of BolaZ, which mainly includes three aspects: the extra RTT brought by P-APIs cache resource ID; the extra RTT brought by C-APIs query cache; the extra RTT brought by C-APIs query database.

6.5.2 Results. Firstly, we test the performance cost of the P-API cache resource ID. To restore the real application scenario, we use Apache JMeter [21] to simulate 100 users accessing multiple P-APIs concurrently at different data levels and average the RTT. The test results are shown in Fig.15. As the amount of data in the database increases, the performance cost of BolaZ increases. BolaZ only increases the delay by hundreds of milliseconds, which users can accept.



(a) Blog (b) NewbeeMall (c) Mall

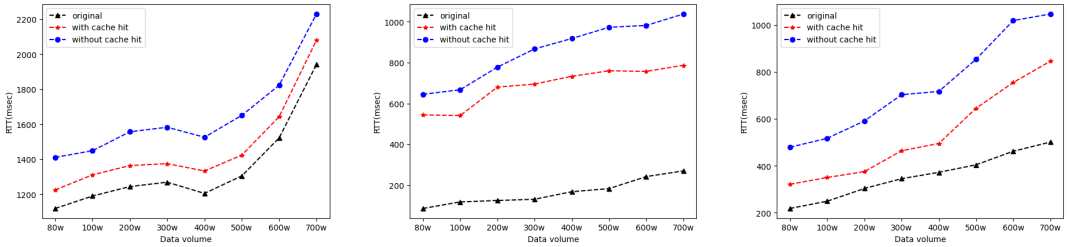
Fig. 15. Performance overhead of P-API

Second, we test the performance overhead caused by BolaZ's MSG interval checking of C-API. We first tested the average RTT of C-API at different data levels under 1,000 user concurrency in the original system. Then, we tested the average RTT of system cache hits and misses under BolaZ protection. The test results are shown in Fig.16. Through experiments, we found that 1) The delay caused by BolaZ to the original system remains within 1 second whether it is a cache hit or not. 2) The cache hit query speed is faster than the direct query database, but the gap is not very big. The gap is small because BolaZ adds the primary key resource ID as a query condition to the SQL statement. After testing in Mysql8, the query speed of SQL statements with primary key conditions can be maintained at the millisecond level at the 700w data level.

The performance overhead of BolaZ is due to the query cache or database, independent of original systems. The overhead is entirely related to the speed of cache or database queries.

Answer to RQ2: Regardless of P-APIs cache resource IDs or C-APIs query resource IDs from the cache and database, the performance cost of BolaZ remains at the millisecond level.

Just Accepted



(a) Blog (b) NewbeeMall (c) Mall

Fig. 16. Performance overhead of BolaZ

Table 7. Evaluation on BOLA vulnerabilities

Type	API Type	API
BFLA	Vertical Privilege	GET /squareComment/getAllSquare/{page}/{limit}
	CVE-2023-36100(C-API)	POST /api/User/ChangeUser/{jwt} Body:userId
	P-API	GET /User/GetUserInfoByid/{user_self_id}
BOPLA	Excessive Data Exposure	GET /api/share/treeList
	CVE-2023-32310(C-API)	POST /api/share /removePanelShares/{panelId}
	P-API	POST /api/share/shareOut
UASBF	CVE-2023-32310(C-API)	POST /api/sys_msg/batchDelete Body:msgId
	P-API	POST /api/sys_msg/list/{goPage}/{pageSize}

6.6 RQ3:How Effective is BolaZ's Defense Against Real-world BOLA Attacks?

6.6.1 Setup. BolaZ determines the MSG interval of the BOLA attack injection point by analyzing the source code of the vulnerable systems corresponding to CVE-2023-36100 and CVE-2023-32310. It then deploys these vulnerable systems and accesses the API using resource IDs outside the MSG interval, thereby detecting BOLA vulnerabilities.

6.6.2 Results. The results of the evaluation on real-world BOLA vulnerabilities are shown in Table 7.

BFLA(CVE-2023-36100). The attacker obtains the userids of all users by accessing API1*(GET /squareComment/getAllSquare)* vertically unauthorizedly and then tampering with the *userId* injection point of C-API1*(POST /api/User/ChangeUser Body:User,userId)* to modify the information of other users. By analyzing the data flow, BolaZ found that P-API1*(GET /User/GetUserInfoByid/{user_self_id})* provided the userId for C-API1. P-API1 returns the user's own userId, so C-API1 can only use the user's own userId.

BOPLA(CVE-2023-32310). the attacker use API2*(GET /api/share/treeList)* can view the panelId of the panels shared by other users and modify the panelId parameter of C-API2*(POST /share/removePanelShares/{panelId})* to remove the panels shared by other users. However, under the protection of BolaZ, attackers do not have permission to use panelids shared by other users in C-API2. PanelId authorization fails because there is no data flow in the system for API2 to propagate panelId to C-API2. P-API2*(POST /share/shareOut)* is associated with C-API2, and P-API2 isolates the panelId of C-API2 into the paneId of the panel that the user shares.

UASBF(CVE-2023-32310). The msgId injection point of C-API3*(POST /sys_msg/batchDelete Body:msgId)* is a numeric type, and the attacker constantly guesses msgId of other users. BolaZ learns from the data stream that

Just Accepted

P-API3 provides msgIds for C-API3. P-API3*(POST /sys_msg/list)* generates the system messages users receive, so C-API3 can only delete messages using the msgId of the messages it receives.

Detection in the wild. We use BolaZ to scan projects in Table 3 to obtain the MSG interval of the C-API resource ID. By passing the resource ID outside the permission into the C-API parameter, we determine whether there is a vulnerability according to the response result. BolaZ reports a total of 36 vulnerabilities, of which 35 vulnerabilities have been manually confirmed as real vulnerabilities, as shown in Table 8. The false positive is caused by the modification of the resource ID during the propagation process. We have submitted vulnerability information to the CVE vulnerability database. Due to ethical considerations, we will disclose the vulnerability after notifying the manufacturer to repair it. The details of BolaZ report vulnerabilities are shown in Table 9.

We also found an implementation-defective API (API1: *findartbyuserid/{userId}*) whose function is to obtain a self-published article. The *userId* parameter in API1 is the user's own userId, but the caller can obtain the article published by other users by modifying the *userId*. API1 does not perform access control checks on *userId*. Although there is another API in the system that can view all user-published articles, developers still need to add access control checks to API1 during implementation.

Table 8. BOLA vulnerabilities reported by BolaZ

Project	SELECT		INSERT		UPDATE		DELETE		Total	
	TP	FP	TP	FP	TP	FP	TP	FP	TP	FP
Blog	0	0	0	0	0	0	0	0	0	0
BookStore	2	0	3	0	3	0	3	0	11	0
Mall	1	0	0	0	1	0	0	0	2	0
NewbellMall	0	0	0	0	0	0	0	0	0	0
IceCms	1	0	3	0	0	0	0	0	4	0
MusicwWbsite	3	0	4	0	3	0	3	0	13	0
OnlineExam	1	0	1	1	1	0	0	0	3	1
UniversityForum	0	0	2	0	0	0	0	0	2	0
InformationSystem	0	0	0	0	0	0	0	0	0	0
OnlineMall	0	0	0	0	0	0	0	0	0	0
Total	8	0	13	1	8	0	6	0	35	1

6.6.3 Case study of unknown vulnerabilities. We use three BOLA vulnerabilities identified by BolaZ to show the practical utility of BolaZ in real-world scenarios. Since the vulnerability has not yet been disclosed, we hide the CVE number and use the pseudo-code to illustrate the vulnerability.

Select Case. As shown in Listing 5. In the C-API(/orderdetail/{orderId}), an attacker illegally obtains the order information of other users by passing in the *orderId* of other users. By analyzing the production and propagation of orderId, BolaZ found that P-API (/myOrder) provided *orderId* for C-API. The P-API can only obtain the *orderId* created by userself, thereby detecting the BOLA vulnerability.

Listing 5 Vulnerability pseudocode of select case

```
1: //C-API:/orderdetail/{orderId}
2: public order getOrder(Integer orderId){
3:   Order order= orderService.get(orderId);
4: }
5: //P-API:/myOrder
```

Just Accepted

```
6: public List<Order> getMyOrder(){
7:   return orderService.getMyOrder(getCurrentUserId());
8: }
```

Update and Delete Case. As shown in Listing 6. Since the primary key *aid* of the address table is an integer type, an attacker can call C-API1 and C-API2 by guessing *aid* to delete and update the address resources, respectively. BolaZ found that P-API provides *aid* for C-API1 and C-API2, while P-API can only read the user's own address resources. Therefore, BolaZ limits the *aid* in C-API1 and C-API2 to the user's own addressID interval.

Listing 6 Vulnerability pseudocode of delete/update case

```
1: //C-API1:/delUserAddress/{aid}
2: public void delete(Integer aid){
3:    addService.delete(aid);
4: }
5: //C-API2:/modifyUserAddress body:Address(aid,addr)
6: public void update(Address add){
7:   addService.update(add);
8: }
9: //P-API:/userAddress
10: public List<Address> getMyAdd(){
11:   return addService.getMyAdd(getCurrentUserId());
12: }
```

Answer to RQ3: BolaZ can defend against BOLA attacks in three attack modes and supports the detection of BOLA vulnerabilities in real-world projects.

6.7 RQ4:How does BolaZ Compare with the SOTA Approach?

6.7.1 Setup. BOLARAY first analyzes the source code to infer the authorization model of resources, and then verifies whether these models correctly implement access control checks. BOLARAY determines the access control model of the resource based on the artificially summarized authorization mode. BolaZ does not need to classify resources into specific authorization models. Therefore, we focus on the difference between the authorization model of BOLARAY and the access control model of BolaZ based on system logic. To eliminate bias introduced by the BOLARAY reasoning resource authorization model, we first manually labeled the authorization models of the resources. Since the authorization model is quite common, the manual labeling is feasible with moderate efforts. Secondly, after determining the authorization model for the resources, we can identify which resources users have operational permissions for and which they do not. Therefore, we directly determine whether the API has performed access control checks through dynamic API testing[2].

BOLARAY needs to analyze *admincolumn* to determine administrator permissions, but PHP and SpringBoot have different methods for determining administrator permissions. In some SpringBoot APIs, there is no administrator permission flag. BolaZ also detected APIs with non-administrator permissions. Therefore, in the comparative experiment, we remove the APIs of administrator permissions, registration and login, and reduced the false positives caused by BOLARAY due to *admincolumn*.

[2]Code availability: https://github.com/wuanbin/bolaz/blob/main/bolaray/bolaray-api-test.py

Just Accepted

Table 9. Identified unpublished vulnerabilities in projects

Project	Type	Description
Mall	SELECT	Attackers get order informations of other users by modifying the orderId.
Mall	UPDATE	Attackers update order informations of other users by modifying the orderId.
Musicwebsite	UPDATE	Attackers update info of other users by modifying the userId.
Musicwebsite	UPDATE	Attackers update password of other users by modifying the userId.
Musicwebsite	UPDATE	Attackers update avatar of other users by modifying the userId.
Musicwebsite	SELECT	Attackers get collection's detail of other users by modifying the userId.
Musicwebsite	INSERT	Attackers add collection's of other users by modifying the userId.
Musicwebsite	DELETE	Attackers delete collection's of other users by modifying the userId.
Musicwebsite	SELECT	Attackers get collection's status of other users by modifying the userId.
Musicwebsite	SELECT	Attackers get rank of other users by modifying the userId.
Musicwebsite	INSERT	Attackers add rank of other users by modifying the userId.
Musicwebsite	INSERT	Attackers add comments of other users by modifying the userId.
Musicwebsite	DELETE	Attackers delete comments of other users by modifying the commentId.
Musicwebsite	INSERT	Attackers add support of other users by modifying the commentId and userId.
Musicwebsite	DELETE	Attackers delete support of other users by modifying the commentId and userId.
IceCMS	SELECT	Attackers get other users informations by modifying the userId.
IceCMS	INSERT	Attackers insert other users article comment by modifying the userId.
IceCMS	INSERT	Attackers insert other users square comment by modifying the userId.
IceCMS	INSERT	Attackers insert other users resource comment by modifying the userId.
BookStore	DELETE	Attackers delete addresses of other users by modifying the addressId.
BookStore	UPDATE	Attackers update addresses of other users by modifying the addressId.
BookStore	INSERT	Attackers add cart of other users by modifying the account.
BookStore	DELETE	Attackers delete cart of other users by modifying the account.
BookStore	DELETE	Attackers batch delete cart of other users by modifying the account.
BookStore	UPDATE	Attackers update cart of other users by modifying the account.
BookStore	SELECT	Attackers get cart of other users by modifying the account.
BookStore	INSERT	Attackers init order of other users by modifying the account.
BookStore	INSERT	Attackers add order of other users by modifying the account.
BookStore	SELECT	Attackers get order of other users by modifying the account.
BookStore	UPDATE	Attackers update order status of other users by modifying the orderId.
OnlineExam	UPDATE	Attackers update password of other users by modifying the studentId.
OnlineExam	INSERT	Attackers add score of other users by modifying the studentId.
OnlineExam	SELECT	Attackers get score of other users by modifying the studentId.
University Forum	INSERT	Attackers add post of other users by modifying the userId.
University Forum	INSERT	Attackers add comment of other users by modifying the userId.

Just Accepted

6.7.2 Results. The results of this comparison are summarized in Table 10. BOLARAY correctly identified 27 true BOLA vulnerabilities, all of which were also disclosed by BolaZ. However, BOLARAY missed 8 vulnerabilities reported by BolaZ because it cannot identify BOLA vulnerabilities from SELECT statements. In contrast, BolaZ can analyze BOLA vulnerabilities with all types of SQL statements. BOLARAY reported 3 false vulnerabilities, but they are different from those reported by BolaZ, as shown in Table 11.

6.7.3 Case study of false positives. Through an in-depth investigation, we have summarized the causes of these false positives as follows.

Table 10. BOLA vulnerabilities reported by BOLARAY

Project	SELECT		INSERT		UPDATE		DELETE		Total	
	TP	FP	TP	FP	TP	FP	TP	FP	TP	FP
Blog	0	0	0	0	0	0	0	0	0	0
BookStore	0	0	3	0	3	0	3	0	9	0
Mall	0	0	0	0	1	0	0	0	1	0
NewbellMall	0	0	0	0	0	0	0	0	0	0
IceCms	0	0	3	0	0	0	0	0	3	0
MusicwWbsite	0	0	4	0	3	1	3	0	10	1
OnlineExam	0	0	1	0	1	1	0	1	2	2
UniversityForum	0	0	2	0	0	0	0	0	2	0
InformationSystem	0	0	0	0	0	0	0	0	0	0
OnlineMall	0	0	0	0	0	0	0	0	0	0
Total	0	0	13	0	8	2	6	1	27	3

Case 1: Application-level Authorization [18]. BOLARAY simplified the application-layer access control policy with the administrator role, and this simplification resulted in 2 false positives. In the online exam project, the teacher has the permission to modify, and delete all user data, but BOLARAY believes that non-administrator users can only operate on their own user data, resulting in false positives.

Case 2: Column-level Authorization [18]. Some applications require the authorization model to be more granular and limited to specific columns, leading to a false positive. In the music website project, the comment table is defined by BOLARAY as the ownership model, and a comment can only be updated/deleted by its owner. However, the comment-like API (/comment/like) modifies the comment's *like_count* column, which can be modified by all users.

BOLARAY reported 2 more false vulnerabilities than BolaZ. This is because BOLARAY performs authorization checking based on the artificially summarized authorization model, which will lead to failure in some application scenarios. Its authorization model does not fit the workflow of the system, and wrongly determines the context between APIs. However, BolaZ defense rules based on the system's best-practice authorization logic solve these defects.

Table 11. False positives reported by BOLARAY

Project	API	Description
OnlineExam	PUT /student	Application-level Authorization.
	DELETE /student /studentId	Application-level Authorization.
MusicWebsite	POST /comment/like	Column-level Authorization.

Insight #2: Combined with BOLARAY's authorization models, BolaZ can alleviate false negatives caused by "client logic control (detailed in 3.4.2)".

Answer to RQ4: Compared with BOLARAY, BolaZ not only supports BOLA vulnerability detection for all types of SQL statements, but also breaks the limitations of the artificially summarized authorization model and has a lower false positive rate.

Just Accepted

6.8 RQ5: How does complementing resource ID propagation at different stages contribute to the success of BolaZ?

6.8.1 Setup. Tracing the process of resource ID propagation solely based on CodeQL is incomplete. BolaZ mainly fills in the data flow of resource ID propagation at three levels. Firstly, at the framework level. If P-API generates MSG intervals for multiple resource IDs and the MSG interval for resource ID_B depends on the MSG interval for resource ID_A, the tracking of resource ID_B will be interrupted.

The second issue is aliasing. During the propagation of resource IDs, aliases, or other copy and assignment operations (e.g., set, copylist) may occur. These types of operations can cause CodeQL to lose track of the resource ID propagation. BolaZ connects the breakpoints through the rule library. If alias/field propagation is unanalyzed, BolaZ will also fail to identify the injection point and MSG interval.

The third issue is the interruption during the association process of P-API and C-API. CodeQL itself does not support the propagation (indirect association) from P-API to router/event and then to C-API. BolaZ has completed the interruption at the router/event when associating resource IDs between P-API and C-API. If the identification of the indirect association is cancelled, the MSG interval identification will fail.

As for string operations on resource IDs (e.g., concatenation, extraction), CodeQL supports the propagation of string operations. However, the premise of BolaZ is that the resource ID remains unchanged during the propagation process. We further verify the premise of BolaZ by analyzing the experimental results.

Table 12. Framework abstraction ablation experiment, IP denotes inject point

Project	IP-Pr	IP-Re	MSG-Pr	MSG-Re
NewbeeMall	15/15	15/15	12/12	12/15
OnlineExam	20/20	20/20	17/18	17/20
Blog	15/15	15/16	10/10	10/16
UniversityForum	7/7	7/7	4/4	4/7
InformationSystem	2/2	2/2	2/2	2/2

6.8.2 Result. As shown in Table 12, after removing the association analysis between multiple SQLs at the framework level. The identification of the BOLA attack injection point has not been affected; the P-API cannot effectively identify the MSG interval of the sub-resource, resulting in the MSG interval correlation failure of the downstream C-API injection point. The recall for identifying the MSG interval has decreased to 76.93%. It decreased by 7.75% compared with the original one.

Table 13. Alias/field propagation ablation experiment, IP denotes inject point

Project	IP-Pr	IP-Re	MSG-Pr	MSG-Re
NewbeeMall	15/15	15/15	3/3	3/15
OnlineExam	20/20	20/20	17/18	17/20
Blog	15/15	14/16	12/12	12/16
UniversityForum	7/7	4/7	1/1	1/7
InformationSystem	2/2	2/2	2/2	2/2

As shown in Table 13, if the analysis of alias/field propagation is canceled, there may be interruptions in the process of identifying the injection point and generating the MSG interval. The recall for identifying attack

Just Accepted

Table 14. P-API and C-API associated ablation experiments, IP denotes inject point

Project	IP-Pr	IP-Re	MSG-Pr	MSG-Re
NewbeeMall	15/15	15/15	1/1	1/15
OnlineExam	20/20	20/20	6/6	6/20
Blog	15/15	14/16	0/0	0/16
UniversityForum	7/7	4/7	0/0	0/7
InformationSystem	2/2	2/2	0/0	0/2

injection points drops to 73.13%, a decrease of 25.62% compared to the original. The recall for identifying MSG intervals drops to 58.86%, a decrease of 25.82% compared to the original.

As shown in Table 14, if the association between P-API and C-API at the router/event is removed, the identification of injection points will not be affected either. However, the recall of identifying MSG intervals has decreased to 7.33%, and in some projects, it has even reached 0. By analyzing the API association, it can be found that the resource ID is mainly used to identify resources when implementing API CRUD operations, and CRUD operations are basically triggered by events or routers. The indirect association between APIs is an important part of maintaining the integrity of the resource ID propagation data flow.

For the string operations on resource IDs, we analyzed the CodeQL logs and found that no string operations are involved in the process of generating resource IDs from the SQL layer of the P-API to their consumption in the SQL layer of the C-API. When the resource IDs were passed to the cache, there were string concatenation operations, but this did not affect BolaZ's identification of injection points and MSG intervals.

Answer to RQ5: The completeness of the resource ID propagation data stream is directly related to the accuracy of injection point identification and MSG interval identification. The more accurate the data stream, the more effective it is to reduce the false positives and false negatives of the injection point's MSG interval.

7 Discussion and Future Work

7.1 Threat to Validity.

Through our experiments, we identified certain limitations in BolaZ's data flow analysis when the scenarios are relatively specific. First, as systems grow more complex in functionality and logic, it becomes challenging to interpret client-side control conditions using static code analysis. BolaZ sometimes was unable to determine which resource IDs produced by P-APIs are passed to C-APIs solely by analyzing condition code, particularly in scenarios involving client-controlled deletion and modification operations.

Second, due to varying levels of developer expertise and coding practices, the same logic can be implemented in numerous ways. CodeQL cannot accommodate all coding patterns, leading to interruptions in some data flow analysis processes. As a result, API identification and association may fail, leaving some BOLA vulnerabilities unaddressed. In these cases, BolaZ will not restrict such APIs, and the vulnerabilities may persist.

Third, BolaZ is currently implemented on the SpringBoot framework. A fact is that the propagation logic and behavior of resource IDs remains consistent in varied languages or frameworks. Hence, BolaZ is a generic approach at the methodological level.

Fourth, the resource ID may also change under certain special circumstances. In such scenarios, BolaZ is not supported. However, the logic followed by BolaZ for capturing the propagation of resource IDs remains universal. Even if the resource ID changes, addressing the BOLA vulnerability transforms into determining the logic of the change in the resource ID.

Just Accepted

7.2 Future Work.

First, BolaZ currently only supports databases. In the future, we plan to investigate the potential of the MSG concept in other types of storage systems. Second, improvements are needed in BolaZ's front-end code analysis. We aim to integrate dynamic application security testing to better capture the logical relationships between P-APIs and C-APIs. Additionally, to address data flow interruptions in taint tracking, we will offer open programming interfaces, allowing users to adapt BolaZ to their system's coding patterns through customized rules.

8 Related Work

8.1 API Automated Testing

Most automated tests for RESTful APIs focus on functional tests and are not used to detect BOLA vulnerabilities. However, we can draw ideas from API automated testing methods.

RestTestGen [45] analyzed the producers and consumers of resources in the API based on OAS. RESTler [3] deduces producer-consumer dependencies of resources and resource IDs based on OAS. EvoMaster [2] generates test cases for RESTful APIs by analyzing the source code. However, EvoMaster believes that users can only operate on the resources they create, which does not apply to all application scenarios. Atlidakis et al. [4] used automated API testing to verify whether the API violated the user-namespace rule. The rule holds that the resources produced by user A cannot be accessed by user B, and does not apply to all resource relationships. Corradini et al. [10] Detect Mass Assignment Vulnerabilities in RESTful APIs using automated black-box testing. The method using common naming practices to identify the Resource ID is inaccurate. RESTest [28] is an automated black-box testing tool for RESTful APIs, which supports inference of dependencies between parameters.

8.2 Static Anlaysis-based BOLA Vulnerability Detection

Application source code contains resource access patterns, so many tools use static analysis techniques to infer resource's authorization rules from the source code.

SPACE [32] requires developers to provide a mapping of application resources to the basic types that occur in SPACE catalog. SPACE uses symbolic execution to extract the data exposures from source code. Then SPACE checks whether each data exposure is allowed. Cancheck [5] also requires developers to provide authorization rules for resources. RoleCast [37] common software engineering patterns to determine resource authorization patterns, but RoleCast's patterns do not apply to all web applications. MACE [29] analyzes INSERT statements to determine relationships between users and resources, but MACE only supports the ownership model [18] and cannot detect the BOLA vulnerability of SELECT statements. BOLARAY [18] combines SQL and static analysis to automatically identify BOLA vulnerabilities in database-backed applications. However, BOLARAY also requires developers to mark DAL specifications, and does not support the detection of BOLA vulnerabilities in SELECT statements. The principle of MOCGuard [26] and BOLARAY to detect BOLA vulnerabilities is highly similar. They are based on human-summarized authorization models and use the relationship between the database tables corresponding to the resources to infer whether the user has permission to access the corresponding resources. The four authorization models summarized by BOLARAY are more comprehensive than MOCGuard and cover more scenarios. Similarly, MOCGuard also cannot handle the dynamic authorization logic of unknown systems due to the static nature of the artificially summarized authorization model.

8.3 Runtime Monitoring

There are also some works to enforce access control at system runtime. Nemesis [14] ensures that only authenticated users can access resources under the manual authorization policy. FlowWatcher [31] found that the access control pattern of resources in most web applications is similar. Therefore, FlowWatcher requires developers to provide an access policy to detect whether the policy is violated during system runtime.

Just Accepted

9 Conclusions

This paper proposes a defense framework for BOLA attacks, BolaZ, based on zero trust. The framework uses static taint tracking technology to obtain propagation data flows of resource IDs and determines APIs' role according to the production and consumption of resource IDs. To capture the context semantic relationship between P-APIs and C-APIs, BolaZ also tracks the propagation path of resource IDs between APIs. Experiments show that BolaZ can effectively identify and correlate APIs, low-performance overhead, and high security in multi-API vulnerability scenarios.

Acknowledgments

This work is supported by the National Natural Science Foundation of China under Grant 62372323.

Ethics considerations

First, the open-source projects we collect from Github do not involve the privacy of companies or individuals. Secondly, we download the code to the local experiment, and will not affect other companies or individuals. We submitted the discovered vulnerabilities to the CVE platform and notified the relevant project leaders. After the vulnerability is fixed, we will disclose the CVE information.

References

[1] akamai. 2024. owasps-top-10-api-security-risks. *akamai* (2024). https://www.akamai.com/site/zh/documents/white-paper/2023/owasps-top-10-api-security-risks.pdf
[2] Andrea Arcuri. 2019. RESTful API automated test case generation with EvoMaster. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 1 (2019), 1–37. Publisher: ACM New York, NY, USA.
[3] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. Restler: Stateful rest api fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 748–758.
[4] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2020. Checking security properties of cloud service REST APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 387–397.
[5] Ivan Bocic and Tevfik Bultan. 2016. Finding access control bugs in web applications with CanCheck. *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2016), 155–166. https://api.semanticscholar.org/CorpusID:2890296
[6] boylegu. 2023. InformationSystem. https://github.com/boylegu/SpringBoot-vue
[7] Mark Campbell. 2020. Beyond zero trust: Trust is a vulnerability. *Computer* 53, 10 (2020), 110–113. Publisher: IEEE.
[8] CodeQL. 2024. CodeQL. https://github.com/github/codeql
[9] CodeQL. 2024. global taint tracking. https://codeql.github.com/docs/codeql-language-guides/analyzing-data-flow-in-java/
[10] Davide Corradini, Michele Pasqua, and Mariano Ceccato. 2023. Automated black-box testing of mass assignment vulnerabilities in RESTful APIs. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2553–2564.
[11] cp3geek. 2022. universityforum. https://github.com/cp3geek/universityforum
[12] CVE. 2023. *CVE-2023-32310*. Technical Report. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-32310
[13] CVE. 2023. *CVE-2023-36100*. Technical Report. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-36100
[14] Michael Dalton, Christos Kozyrakis, and Nickolai Zeldovich. 2009. Nemesis: Preventing Authentication & Access Control Vulnerabilities in Web Applications. In *USENIX Security Symposium*. https://api.semanticscholar.org/CorpusID:15907882
[15] Dick Hardt. 2012. The OAuth 2.0 Authorization Framework. https://tools.ietf.org/html/rfc6749
[16] Roy Thomas Fielding. 2000. *Architectural styles and the design of network-based software architectures*. University of California, Irvine.
[17] Mahmoud Ghorbanzadeh and Hamid Reza Shahriari. 2020. ANOVUL: Detection of logic vulnerabilities in annotated programs via data and control flow analysis. *IET Information Security* 14, 3 (2020), 352–364. Publisher: Wiley Online Library.
[18] Yongheng Huang, Chenghang Shi, Jie Lu, Haofeng Li, Haining Meng, and Lian Li. 2024. Detecting Broken Object-Level Authorization Vulnerabilities in Database-Backed Applications. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2934–2948.
[19] Muhammad Idris, Iwan Syarif, and Idris Winarno. 2021. Development of vulnerable web application based on OWASP API security risks. In *2021 International Electronics Symposium (IES)*. IEEE, 190–194.
[20] Samuel Jero, Juliana Furgala, Runyu Pan, Phani Kishore Gadepalli, Alexandra Clifford, Bite Ye, Roger Khazan, Bryan C Ward, Gabriel Parmer, and Richard Skowyra. 2021. Practical principle of least privilege for secure embedded systems. In *2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 1–13.

Just Accepted

[21] Jmeter. 2024. Jmeter. https://jmeter.apache.org/
[22] JWT. 2015. JSON Web Token. http://www.rfc-editor.org/rfc/rfc7519
[23] Sreejith Keeriyattil and Sreejith Keeriyattil. 2019. Microsegmentation and zero trust: Introduction. *Zero Trust Networks with VMware NSX: Build Highly Secure Network Architectures for Your Data Centers* (2019), 17–31. Publisher: Springer.
[24] Junhyoung Kim, TaeGuen Kim, and Eul Gyu Im. 2014. Survey of dynamic taint analysis. In *2014 4th IEEE International Conference on Network Infrastructure and Digital Content*. 269–272. doi:10.1109/ICNIDC.2014.7000307
[25] libhunt. 2025. Open-source projects categorized as spring-boot. https://www.libhunt.com/topic/spring-boot
[26] Fengyu Liu, Youkun Shi, Yuan Zhang, Guangliang Yang, Enhao Li, and Min Yang. 2025. MOCGuard: Automatically Detecting Missing-Owner-Check Vulnerabilities in Java Web Applications. In *2025 IEEE Symposium on Security and Privacy (SP)*. 903–919. doi:10.1109/SP61157.2025.00010
[27] macrozheng. 2024. mall. https://github.com/macrozheng/mall
[28] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2020. RESTest: Black-box constraint-based testing of RESTful web APIs. In *Service-Oriented Computing: 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates, December 14–17, 2020, Proceedings 18*. Springer, 459–475.
[29] Maliheh Monshizadeh, Prasad Naldurg, and VN Venkatakrishnan. 2014. Mace: Detecting privilege escalation vulnerabilities in web applications. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 690–701.
[30] MQPearth. 2024. Blog. https://github.com/MQPearth/Blog
[31] Divya Muthukumaran, Dan O'Keeffe, Christian Priebe, David Eyers, Brian Shand, and Peter Pietzuch. 2015. FlowWatcher: Defending against data disclosure vulnerabilities in web applications. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 603–615.
[32] Joseph P Near and Daniel Jackson. 2016. Finding security bugs in web applications using a catalog of access control patterns. In *Proceedings of the 38th International Conference on Software Engineering*. 947–958.
[33] newbee-ltd/. 2024. newbee-mall. https://github.com/newbee-ltd/newbee-mall-api
[34] OWASP API Security. 2023. OWASP API Security Project. https://owasp.org/www-project-api-security/
[35] Mayra Samaniego and Ralph Deters. 2018. Zero-trust hierarchical management in IoT. In *2018 IEEE international congress on Internet of Things (ICIOT)*. IEEE, 88–95.
[36] Sailik Sengupta, Ankur Chowdhary, Abdulhakim Sabur, Adel Alshamrani, Dijiang Huang, and Subbarao Kambhampati. 2020. A survey of moving target defenses for network security. *IEEE Communications Surveys & Tutorials* 22, 3 (2020), 1909–1941. Publisher: IEEE.
[37] Sooel Son, Kathryn McKinley, and Vitaly Shmatikov. 2011. RoleCast: Finding Missing Security Checks When You Do Not Know What Checks Are. In *Sigplan Notices - SIGPLAN*, Vol. 46. 1069–1084. doi:10.1145/2076021.2048146
[38] sourcecodeexamples. 2025. sourcecodeexamples. https://www.sourcecodeexamples.net/
[39] spring-projects. 2025. spring-boot dependency graph. https://github.com/spring-projects/spring-boot/network/dependents
[40] StuHaiBin. 2020. BookStore. https://github.com/StuHaiBin/bookStore-Springboot-Vue
[41] tablu666. 2020. onlinemall. https://github.com/tablu666/springboot-vue-online-mall
[42] Gonçalo André Carneiro Teixeira. 2023. Security Testing of Web APIs. (2023).
[43] telerik. 2024. Fiddler. https://www.telerik.com/fiddler-b
[44] Thecosy. 2024. IceCMS. https://github.com/Thecosy/IceCMS
[45] Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2020. Resttestgen: automated black-box testing of restful apis. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 142–152.
[46] Yin-Hongwei. 2024. music-website. https://github.com/Yin-Hongwei/music-website
[47] YXJ2018. 2024. SpringBoot-Vue-OnlineExam. https://github.com/YXJ2018/SpringBoot-Vue-OnlineExam
[48] Man Zhang, Bogdan Marculescu, and Andrea Arcuri. 2021. Resource and dependency based test case generation for RESTful Web services. *Empirical Software Engineering* 26, 4 (2021), 76. Publisher: Springer.

Just Accepted